

UNIT - I

Q1. Program Concept

A **program** is a set of instructions written in a programming language to perform a specific task. Programs are designed to solve problems, automate tasks, and enhance efficiency in computations. Every program follows a sequence of instructions that the computer executes step by step.

Characteristics of a Good Program:

1. **Correctness:** The program must produce the correct output for given inputs.
2. **Efficiency:** It should use minimal system resources (CPU, memory, etc.).
3. **Readability:** Code should be easy to understand with proper indentation and comments.
4. **Maintainability:** It should be easy to modify and update when required.
5. **Portability:** A good program should run on different operating systems with minimal modifications.

Example:

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, World!";
    return 0;
}
```

Q2. Analysis of Algorithm

Algorithm analysis helps determine the efficiency of an algorithm by considering its **time complexity** and **space complexity**.

1. Time Complexity

It measures how the runtime of an algorithm increases as the input size grows. It is expressed in **Big-O notation**, such as:

- $O(1)$ - Constant time
- $O(\log n)$ - Logarithmic time
- $O(n)$ - Linear time
- $O(n^2)$ - Quadratic time
- $O(2^n)$ - Exponential time

Example: Finding the sum of first N natural numbers using a loop (O(n))

```
int sum = 0;
for(int i = 1; i <= n; i++) {
    sum += i;
}
cout << sum;
```

2. Space Complexity

It measures the amount of memory an algorithm requires.

Q3. Flowchart

A **flowchart** is a diagrammatic representation of an algorithm using symbols. It helps in visualizing the flow of execution.

Symbols Used in Flowcharts:

1. **Oval** - Start/End
2. **Parallelogram** - Input/Output
3. **Rectangle** - Process (calculations)
4. **Diamond** - Decision-making (if-else conditions)

Example Flowchart for Adding Two Numbers:

1. Start
2. Input a, b
3. $sum = a + b$
4. Print sum
5. End

Q4. Programming Techniques

1. **Top-Down Approach:** Breaking a problem into subproblems and solving them individually.
2. **Bottom-Up Approach:** Solving smaller problems first and integrating them.
3. **Modular Programming:** Dividing a large program into smaller functions.
4. **Object-Oriented Programming (OOP):** Organizing code using objects and classes.

Example:

```
int add(int a, int b) {
    return a + b;
}
int main() {
    cout << add(5, 3);
}
```

Q5. Introduction to C++

C++ is an **object-oriented programming language** developed by Bjarne Stroustrup. It supports procedural and object-oriented programming paradigms.

Features of C++:

- Object-Oriented Programming (OOP)
- Faster execution compared to interpreted languages
- Rich standard library
- Strongly typed language

Example:

```
#include <iostream>
using namespace std;
class Car {
public:
    void drive() { cout << "Driving a car"; }
};
int main() {
    Car c;
    c.drive();
}
```

Q6. Structure of a C++ Program

A typical C++ program consists of:

1. **Header Files** (#include <iostream>)
2. **Main Function** (int main() { })
3. **Statements** inside the main function
4. **Return Statement** (return 0;)

Q7. Creating, Compiling, Linking, and Executing a C++ Program

Steps:

1. **Writing the Code:** Save as .cpp file.
2. **Compiling:** Converts source code to machine code (g++ program.cpp -o program.exe)
3. **Linking:** Combines object files.
4. **Executing:** ./program.exe

Q8. Tokens in C++

Tokens are the smallest elements in a C++ program:

- **Keywords:** int, return
 - **Identifiers:** sum, age
 - **Literals:** 100, "Hello"
 - **Operators:** +, -
 - **Special Symbols:** {, }
-

Q9. Keywords in C++

Reserved words that have special meaning. C++ has about 95 keywords, such as:

- int, if, else, return, for, while, switch, class, public, private, void, new, delete, etc.

Example:

```
int main() {  
    int num = 10;  
    if (num > 0) {  
        cout << "Positive number";  
    }  
    return 0;  
}
```

Q10. Identifiers in C++

Identifiers are names assigned to variables, functions, classes, and objects.

Rules for Identifiers:

- Must begin with a letter (A-Z or a-z) or underscore (_)
- Cannot be a keyword

- Must not contain special characters like @, \$, %

Example:

```
int age;  
double salary;  
string name;
```

Q11. Constants in C++

Constants are fixed values that do not change during program execution. Types of constants:

- **Integer Constants:** 10, -5, 0
- **Floating Point Constants:** 3.14, -0.99
- **Character Constants:** 'A', '5'
- **String Constants:** "Hello"
- **Boolean Constants:** true, false

Example:

```
const float PI = 3.14;  
cout << "Value of PI: " << PI;
```

Q12. Basic Data Types in C++

In C++, data types define the type of data that can be stored and manipulated within a program. The basic data types in C++ are:

1. **Integer (int):** Stores whole numbers.
2. **Character (char):** Stores single characters.
3. **Floating-Point (float):** Stores decimal numbers with single precision.
4. **Double (double):** Stores decimal numbers with double precision.
5. **Boolean (bool):** Stores true or false values.
6. **Void (void):** Represents absence of a value, used mainly for functions.

Example:

```
#include <iostream>
using namespace std;

int main() {
    int num = 10;
    char letter = 'A';
    float decimal = 3.14;
    bool flag = true;

    cout << "Integer: " << num << endl;
    cout << "Character: " << letter << endl;
    cout << "Float: " << decimal << endl;
    cout << "Boolean: " << flag << endl;
    return 0;
}
```

Q13. User-Defined Data Types in C++

C++ allows users to define their own data types using the following:

1. **Structure (struct):** Groups different types of variables under one name.
2. **Class (class):** Similar to struct, but with added functionality like encapsulation.
3. **Enumeration (enum):** Defines a set of named integer constants.
4. **Union (union):** Stores different data types in the same memory location.

Example:

```
struct Student {
    string name;
    int age;
};

int main() {
    Student s1 = {"John", 20};
    cout << "Name: " << s1.name << " Age: " << s1.age;
    return 0;
}
```

Q14. Symbolic Constants in C++

Symbolic constants are fixed values that cannot be altered during program execution. They are defined using:

1. **#define preprocessor directive**
2. **const keyword**

Example:

```
#define PI 3.14159 // Using #define
const int MAX = 100; // Using const
```

Q15. Type Compatibility in C++

Type compatibility ensures correct data type usage in expressions. It includes:

- **Implicit Type Conversion** (Automatic conversion)
- **Explicit Type Conversion** (Type Casting)

Example:

```
int x = 10;
double y = x; // Implicit conversion

double a = 5.5;
int b = (int)a; // Explicit conversion
```

Q16. Reference Variables in C++

A reference variable is an alias for another variable.

Example:

```
int x = 10;
int &ref = x;
ref = 20; // Changes x as well
```

Q17. Operators in C++

Operators perform operations on variables and values:

- ***Arithmetic** (+, -, *, /, %)
 - **Relational** (==, !=, <, >, <=, >=)
 - **Logical** (&&, ||, !)
 - **Bitwise** (&, |, ^, <<, >>)
 - ***Assignment** (=, +=, -=, *=, /=, %=)
-

Q18. Functions in C++

Functions modularize code into reusable blocks.

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

Q19. The Main Function in C++

The main() function is the entry point of execution.

```
int main() {  
    cout << "Hello, World!";  
    return 0;  
}
```

Q20. Function Prototyping in C++

Function prototypes declare functions before defining them.

```
int multiply(int, int);  
  
int main() {  
    cout << multiply(4, 5);  
    return 0;  
}  
  
int multiply(int a, int b) {  
    return a * b;  
}
```

Q21. Call by Reference and Call by Value in C++

- **Call by Value:** Passes a copy of the argument.

```
void modify(int &x) { x = 20; }
```

Q22. Inline Functions in C++

Inline functions reduce function call overhead.

```
inline int square(int x) {  
    return x * x;  
}
```

Q23. Default and Const Arguments in C++

- **Default Arguments:** Assign default values to parameters.
- **Const Arguments:** Prevent modification inside a function.

```
void greet(string name = "Guest") {  
    cout << "Hello, " << name;  
}
```

Q24. Function Overloading in C++

Function Overloading allows multiple functions with the same name but different parameters.

```
void print(int a) {  
    cout << "Integer: " << a;  
}  
  
void print(double a) {  
    cout << "Double: " << a;  
}
```